Mini-Shop Web Application

Security Vulnerability Assessment Report

Target application:	Mini-Shop Test Console (local test lab)
Environments tested:	localhost:3000 (baseline build) & localhost:3001 (hardened build)
Assessment type:	Authenticated, manual black-box testing via browser DevTools console
Assessment date:	October 11, 2026
Report date:	October 13, 2026
Scope:	Authentication, authorization (BOLA/IDOR), input validation, XSS, HTTP Parameter Pollution, rate limiting, and admin access controls

Important note on scope: "Mini-Shop" is a deliberately vulnerable practice/lab application (the console is labeled "localhost lab only" and the project ships a `start:vuln` npm script). This report documents findings exactly as observed during testing of the two running instances and is intended for a secure-coding/AppSec training write-up, not a live production audit.

1. Executive Summary

Testing covered two running instances of the same codebase: an intentionally vulnerable baseline on port 3000 and a hardened build on port 3001. Using the application's built-in "Test Console" (a developer panel exposing raw requests to login, register, search, reviews, orders, and admin endpoints), each instance was exercised against the same set of attack payloads: NoSQL/SQL injection strings, reflected/stored XSS payloads, tampered JWTs, HTTP Parameter Pollution, direct object reference (IDOR/BOLA) probing against another user's orders, and brute-force login attempts.

On port 3000, five issues were confirmed as exploitable: broken object-level authorization on the orders endpoint, broken function-level authorization on the admin users endpoint, stored cross-site scripting in the product review feature, HTTP Parameter Pollution on a query-parsing endpoint, and absent login rate-limiting. On port 3001, the equivalent BOLA, admin-access, HPP, and rate-limiting tests all returned correctly restricted results (403/429), indicating these specific issues have already been remediated in that build. The stored XSS payload was not re-tested to completion against 3001 in the captured evidence and should be explicitly re-verified there.

#	Finding	Severity	Port 3000 (baseline)	Port 3001 (hardened)
F 1	Broken Object-Level Authorization (BOLA/IDOR) on /orders/:id	**Critical**	Vulnerable	Fixed
F 2	Broken Function-Level Authorization on /admin/users	**Critical**	Vulnerable	Fixed
F 3	Stored Cross-Site Scripting (XSS) in product reviews	**High**	Vulnerable	Not confirmed retested
F 4	HTTP Parameter Pollution (HPP) on /echo	**Medium**	Vulnerable	Fixed
F 5	No rate limiting on /login (brute-force exposure)	**High**	Vulnerable	Fixed
F 6	Weak registration input validation (empty password accepted)	**Medium**	Vulnerable	Not tested
F 7	NoSQL/SQL injection payloads submitted to /login	**Info**	Rejected (401)	Not tested

Severity ratings follow a standard impact/exploitability scale (Critical / High / Medium / Low / Info) based on OWASP Top 10 (2021) and OWASP API Security Top 10 categories.

2. Methodology

Testing was performed manually from the application's own "Mini-Shop Test Console" UI, which exposes pre-built forms for login, JWT substitution, raw-JSON requests to /login and /register, search and review inputs, order lookups, and one-click admin/HPP/brute-force probes. Browser DevTools (Console and Network panels) were used throughout to capture the exact HTTP method, endpoint, status code, and JSON response body for every request, and to confirm client-side execution of injected payloads (e.g. a triggered `alert()` box).

Tests performed:

- Authentication with valid credentials and capture of the issued JWT
- Submission of a NoSQL injection payload (`{"$ne":null}`) and a classic SQL injection string (`' OR '1'='1' --`) to /login
- Direct, authenticated requests to /orders/:id for order IDs not owned by the logged-in account (BOLA/IDOR)
- Direct, authenticated requests to /admin/users as a non-admin account (broken function-level authorization)
- Reflected/stored XSS payload (`<img src=x onerror=alert('XSS')>`) submitted via search and via the product review form, then viewed on the public reviews page
- HTTP Parameter Pollution against /echo?amount=10&amount=1000 to check duplicate query-parameter handling
- Seven consecutive failed login attempts to test for brute-force rate limiting
- Registration requests with an empty password and with the optional phone field omitted, to test server-side input validation
- Repeat of the above against both the baseline (3000) and hardened (3001) instances for comparison

3. Detailed Findings

F1. Broken Object-Level Authorization (BOLA/IDOR) on /orders/:id

Critical

Affected environment: localhost:3000 (vulnerable); confirmed fixed on localhost:3001 | **Classification:** OWASP API1:2023 Broken Object Level Authorization / CWE-639

Description

After authenticating, requesting GET /orders/<id> for an order ID that did not belong to the logged-in account returned the order record in full, rather than an authorization error. This was reproduced twice: once while authenticated as a user and retrieving an order owned by user_id 102, and again while authenticated as "sara@shop.test" and retrieving order id 1, which belongs to user_id 101 ("ali@shop.test").

Evidence

```
GET /orders/2 → HTTP 200 → {"id":2,"user_id":102,"item":"Phone","amount":200,"paid":0}
```

```
(logged in as sara@shop.test) GET /orders/1 → HTTP 200 →
{"id":1,"user_id":101,"item":"Laptop","amount":300,"paid":0}
```

```
Equivalent request on port 3001: GET /orders/1 → HTTP 403 → {"error":"Forbidden"}
```

```
GET /orders/6 (non-existent id) on 3001 → HTTP 404 → {"error":"Not found"}
```

Impact

Any authenticated user can view another customer's order details, including item purchased, amount, and payment status, simply by changing the numeric ID in the request. In a real deployment this could expose personal purchase history, pricing, and payment state across the entire user base, and if the same pattern extends to order mutation (e.g. a PATCH/PUT or the "pay" flow) could allow one user to mark another user's order as paid or alter it.

Recommendation

Enforce ownership checks server-side on every object lookup: verify `order.user_id == req.user.id` (or an equivalent admin-role check) before returning data, and return 403/404 rather than the record when the check fails — which is exactly the behavior already observed on port 3001, confirming the fix pattern is known and should be applied uniformly to the baseline build and audited across all other object-fetching endpoints.

F2. Broken Function-Level Authorization on /admin/users

Critical

Affected environment: localhost:3000 (vulnerable) | **Classification:** OWASP API5:2023 Broken Function Level Authorization / CWE-862 (Missing Authorization)

Description

GET /admin/users, which should be restricted to accounts with an admin role, returned the full list of registered users (including internal numeric IDs, email addresses, and role assignments) while authenticated as a standard, non-admin account. This was observed on two separate occasions during testing; a third request in between (shown in evidence) correctly returned 403, suggesting the access control is inconsistently applied rather than entirely absent — which is itself a sign of a fragile or role-confused authorization check.

Evidence

```
GET /admin/users → HTTP 200 → [{"id":1,"email":"admin@shop.test","role":"admin"},
{"id":101,"email":"ali@shop.test","role":"user"},
{"id":102,"email":"sara@shop.test","role":"user"}, ...]
```

```
A separate attempt in the same session: GET /admin/users → HTTP 403 → {"error":"Admins only"}
```

```
Repeated later, while authenticated as sara@shop.test: GET /admin/users → HTTP 200 → full user
list returned again
```

Impact

Disclosure of the complete user directory (emails and role assignments) to any logged-in user. Combined with F1, this significantly lowers the bar for account enumeration, targeted phishing, or privilege-related attacks (e.g. identifying which account is the admin account to target next).

Recommendation

Apply a single, centrally-enforced role-check middleware to every admin-prefixed route rather than per-handler checks, and add automated tests that assert non-admin tokens always receive 403 on these routes. Investigate why the check appears to pass intermittently in the current build (e.g. a race condition, a middleware ordering issue, or a tampered/forged JWT role claim being trusted without server-side verification against the user record).

F3. Stored Cross-Site Scripting (XSS) in product reviews

High

Affected environment: localhost:3000 (confirmed exploited); not confirmed re-tested on 3001 | **Classification:** OWASP A03:2021 Injection / CWE-79 (Stored XSS)

Description

The review submission form accepts raw HTML/JS in the review text field with no server-side sanitization or output encoding. A payload of `<img src=x onerror=alert('XSS')>` was submitted via POST /reviews (HTTP 201) and, on loading the public reviews page (/products/1/reviews), executed in the browser, popping a JavaScript alert box reading "XSS". The payload was successfully stored and triggered for multiple product IDs.

Evidence

```
POST /reviews {"productId":1,"text":"<img src=x onerror=alert('XSS')>"} → HTTP 201 → {"ok":true}
```

```
Navigating to localhost:3000/products/1/reviews triggers a browser alert dialog: "localhost:3000
says: XSS"
```

```
Repeated successfully for product IDs 1, 4, and 9
```

Impact

Because the payload is stored and rendered to any visitor of the product reviews page, this is a persistent XSS: it affects every user (including admins) who views the affected product's reviews, not just the attacker. A real payload (rather than a benign alert) could steal session tokens/cookies, perform actions on behalf of the victim, or redirect users to a malicious site.

Recommendation

Encode all user-supplied content on output (e.g. React's default JSX escaping, or an explicit HTML-escaping template helper) and never render review text as raw HTML. Additionally apply server-side sanitization (e.g. an allow-list based sanitizer such as DOMPurify on the server, or stripping HTML entirely for a plain-text review field) and set a Content-Security-Policy header that blocks inline event handlers such as onerror. This finding should be explicitly re-tested against port 3001 to confirm it was addressed alongside the other hardening changes observed there.

F4. HTTP Parameter Pollution (HPP) on /echo

Medium

Affected environment: localhost:3000 (vulnerable); confirmed fixed on localhost:3001 | **Classification:** CWE-235 (Improper Handling of Extra Parameters)

Description

Sending two values for the same query parameter (GET /echo?amount=10&amount=1000) caused the server to parse and return both values as an array rather than rejecting the duplicate or taking a single well-defined value. Depending on which downstream code path consumes a polluted array vs. a scalar, this class of bug can cause validation logic to be bypassed (e.g. if one part of the code validates the first value and another part of the code uses the last value).

Evidence

```
GET /echo?amount=10&amount;=1000 (port 3000) → HTTP 200 →
{"received":["10","1000"],"isArray":true}
```

```
Identical request on port 3001 → HTTP 200 → {"received":"1000","isArray":false}
```

Impact

In this /echo test endpoint the impact is purely diagnostic, but the same duplicate-parameter pattern applied to a real endpoint that handles pricing, quantities, discount codes, or authorization flags could allow an attacker to smuggle a second, unexpected value past validation logic that only inspects the first (or last) occurrence.

Recommendation

Standardize on single-value parameter parsing for parameters that are expected to be scalar (reject or collapse duplicates, as already implemented on port 3001), and explicitly document which parameters are intentionally array-typed so the two cases cannot be confused in review.

F5. No rate limiting on /login (brute-force exposure)

High

Affected environment: localhost:3000 (vulnerable); confirmed fixed on localhost:3001 | **Classification:** OWASP API4:2023 Unrestricted Resource Consumption / CWE-307 (Improper Restriction of Excessive Authentication Attempts)

Description

Seven consecutive failed login attempts against POST /login on port 3000 all returned HTTP 401 with no throttling, lockout, or CAPTCHA challenge. The same automated test against port 3001 was correctly throttled after a small number of attempts, returning HTTP 429 with an explicit "Too many login attempts" message.

Evidence

```
Port 3000, 7x POST /login with wrong password → attempt 1–7: HTTP 401 each time, no blocking observed
```

```
Port 3001, equivalent requests → HTTP 429 → {"error":"Too many login attempts"}
```

Impact

Without rate limiting, the login endpoint on the baseline build is exposed to credential-stuffing and brute-force password guessing attacks at effectively unlimited speed.

Recommendation

Apply the rate-limiting middleware already present on port 3001 to the baseline build as well, and consider layering account-level lockout/backoff and alerting on repeated failures from a single IP or against a single account, in addition to the IP/endpoint-level throttle already confirmed working.

F6. Weak registration input validation

Medium

Affected environment: localhost:3000 | **Classification:** OWASP A04:2021 Insecure Design / CWE-521 (Weak Password Requirements), CWE-20 (Improper Input Validation)

Description

The /register endpoint accepted a new account with an empty-string password (`"password":""`), returning HTTP 201 and a new user ID rather than rejecting the request. A follow-up registration that omitted the phone field entirely was also accepted with 201, and duplicate-email submissions were correctly rejected with 409, showing some validation exists but password strength is not enforced.

Evidence

```
POST /register {"email":"newst","password":"","phone":"03001234567","cnic":"12345-1234567-1"} → HTTP 201 → {"id":106}
```

```
POST /register {"email":"nest","password":"","cnic":"12345-1234567-1"} (no phone field) → HTTP 201 → {"id":107}
```

```
POST /register with an already-registered email → HTTP 409 Conflict
```

Impact

Accounts can be created with no password at all, which (depending on how login compares credentials) could allow trivial authentication bypass, and more generally indicates the registration flow is not enforcing a minimum security bar for new accounts.

Recommendation

Enforce server-side password policy (minimum length/complexity) and required-field validation on /register independent of any client-side form constraints, and add a regression test that an empty or missing password is rejected with 400.

F7. NoSQL / SQL injection payloads submitted to /login

Info

Affected environment: localhost:3000 | **Classification:** OWASP A03:2021 Injection / CWE-89, CWE-943

Description

As a control test, a NoSQL operator-injection payload (`{"password":{"$ne":null}}`) and a classic SQL injection string (`' OR '1'='1' --`) were submitted to POST /login. Both were observed to fail authentication (HTTP 401/429 responses only were captured in this testing session, with no evidence of a successful bypass).

Evidence

```
POST /login {"email":"ali@shop.test","password":{"$ne":null}} → subsequent requests returned HTTP 429 (rate limited) rather than a successful login
```

```
POST /login {"email":"' OR '1'='1' --","password":"x"} → HTTP 401 (Unauthorized)
```

Impact

No evidence of successful injection was captured during this assessment. This is recorded for completeness and audit-trail purposes.

Recommendation

No immediate action required based on observed evidence; recommend confirming with parameterized-query / ORM usage in the login handler's source code (not reviewed as part of this black-box assessment) and adding these payloads to an automated regression/security test suite so the negative result is continuously verified.

---

4. Request/Response Log Summary

The table below summarizes every distinct HTTP exchange captured in the DevTools Console across both environments during this assessment, in testing order.

Env	Method & Endpoint	Status	Notes
3001	POST /login (valid creds)	200	JWT issued
3001	POST /login x4 (rate-limit probe)	429	Throttled as expected
3000	POST /register (duplicate SQLi JSON)	201	{"id":103}
3000	POST /login (valid creds)	200	JWT issued
3000	GET /orders/2	200	BOLA — returned order for user_id 102
3000	GET /echo?amount=10&amount=1000	200	HPP — array returned
3001	POST /reviews (XSS payload)	201	Stored
3001	GET /orders/2	401	Login required (expected, unauthenticated)
3000	POST /login (valid creds)	200	JWT issued
3000	GET /orders/2	200	BOLA reproduced
3000	GET /admin/users	403	Correctly blocked (inconsistent, see F2)
3000	GET /echo?amount=10&amount=1000	200	HPP reproduced
3001	POST /reviews (XSS payload)	201	Stored
3001	GET /orders/2	401	Login required
3001	GET /echo?amount=10&amount=1000	200	HPP fixed — scalar returned
3000	GET /orders/2	200	BOLA reproduced
3000	GET /admin/users	403	Correctly blocked
3001	Register (duplicate) x2	409	Correctly rejected
3001	POST /login (wrong password)	401	Then rate limited

Env	Method & Endpoint	Status	Notes
3001	POST /login (brute force probe)	429	"Too many login attempts"
3001	GET /orders/1 (not owned)	403	BOLA fixed — "Forbidden"
3001	GET /orders/6 (non-existent)	404	"Not found"
3000	GET /orders/4 / /orders/3	404	Not found
3000	GET /admin/users	200	[!] Full user list disclosed (F2)
3000	POST /login (SQLi string in email)	401	Rejected
3000	POST /register (empty password)	201	[!] Accepted (F6)
3000	POST /register (no phone field)	201	[!] Accepted (F6)
3000	POST /reviews (XSS payload) x2	201	Stored for multiple product IDs
3000	GET /orders/1 (as sara@shop.test)	200	[!] Cross-user order disclosed (F1)
3000	GET /orders/2	200	paid:1 — state change observed
3000	GET /admin/users	200	[!] Reproduced again (F2)
3000	POST /login x7 (brute force probe)	401 x7	[!] No rate limiting observed (F5)
3000	GET /echo?amount=10&amount=1000	200	HPP reproduced

5. Prioritized Recommendations

Priority	Finding	Action
1	F1, F2	Port the authorization fixes already present on 3001 (ownership checks on /orders/:id, consistent role-gating on /admin/*) to the 3000 baseline, and add automated authZ regression tests for both.
2	F3	Add output encoding / sanitization to the reviews rendering path and explicitly re-test the stored XSS payload against port 3001.
3	F5	Apply the existing rate-limiting middleware to the 3000 baseline's /login route.
4	F6	Enforce a server-side minimum password policy on /register.
5	F4	Standardize scalar query-parameter parsing across all environments (already fixed on 3001).
6	F7	Add the tested injection payloads to an automated security regression suite to keep the negative result verified over time.

6. Conclusion

The baseline Mini-Shop instance (port 3000) exhibits multiple critical and high-severity issues, most notably broken object- and function-level authorization that allow any authenticated user to read other customers' orders and the full admin user directory, alongside a stored XSS vulnerability in product reviews and an unthrottled login endpoint. The hardened instance (port 3001) demonstrates that fixes for the authorization, HPP, and rate-limiting issues are already implemented and working correctly in the same codebase, which should substantially shorten remediation effort — the primary task is applying the same middleware/checks consistently to the baseline build and completing verification of the XSS fix and the registration validation gap on port 3001.

This report was compiled from a visual review of 28 DevTools screenshots captured during manual testing of the Mini-Shop Test Console. No source code review was performed; all findings are based on observed HTTP request/response behavior.